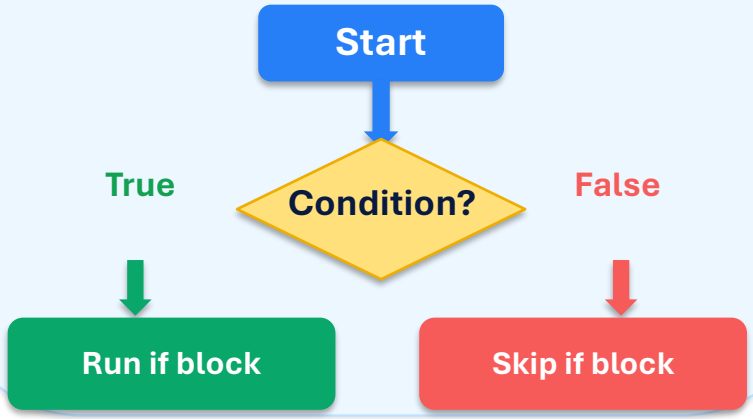
Control Flow and if Statement

How JavaScript makes decisions

1 What is Control Flow?

- Controls the order in which code runs
- Helps programs react to conditions
- Used for decisions, loops, and branching



2 The if Statement

- Checks whether a condition is true
- Runs a block of code only when the condition is true
- Written with parentheses and curly braces

```
if (condition) {
  // code runs here
}
```

3 Example

- Condition: score >= 50
- Action: print 'Pass' if true

```
let score = 75;
if (score >= 50) {
  console.log('Pass');
}
```

Output: Pass

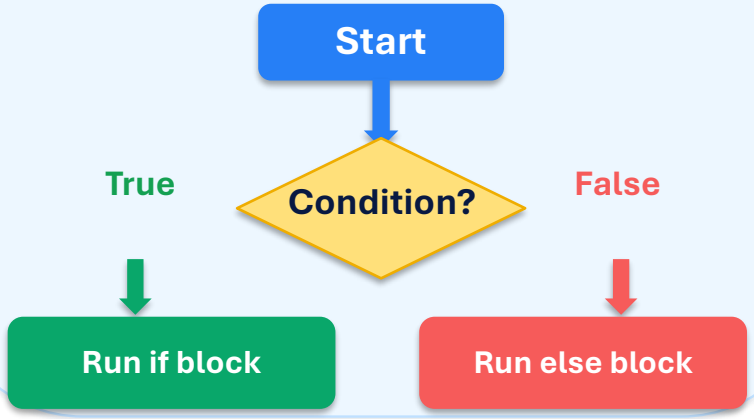
Key Idea | Control flow lets your program choose what to do next.

else Statement

What happens when the if condition is false

1 What is else?

- Runs when the if condition is false
- Provides an alternative path
- Usually follows an if statement



2 Basic Syntax

- Placed after the if block
- Does not need its own condition
- Uses curly braces for its code block

```
if (condition) {
  // code if true
} else {
  // code if false
}
```

3 Example

- Condition: age >= 18
- If false, print 'Too young'

```
let age = 15;
if (age >= 18) {
  console.log('Adult');
} else {
  console.log('Too young');
}
```

Output: Too young

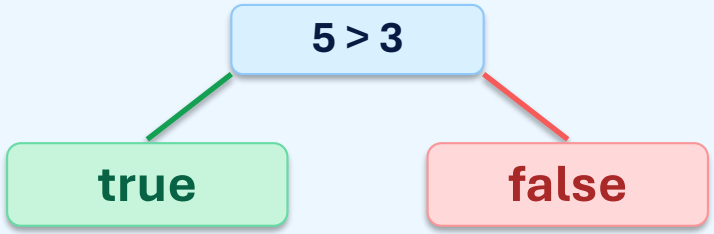
Key Idea | else gives your program a second option when the first condition is not met.

Boolean Expressions

Conditions that evaluate to true or false

1 Definition

- A Boolean expression produces true or false
- Used in if statements and loops
- Often compares values or checks conditions



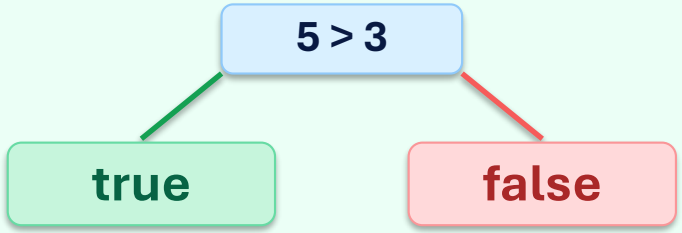
2 Common Examples

- 5 > 3 returns true
- 10 === '10' returns false
- score >= 50 checks a passing condition

```
5 > 3          // true
10 === '10'    // false
score >= 50    // depends
```

3 Why They Matter

- Control decision-making in code
- Help programs react differently
- Form the basis of logic in JavaScript



Result type: Boolean

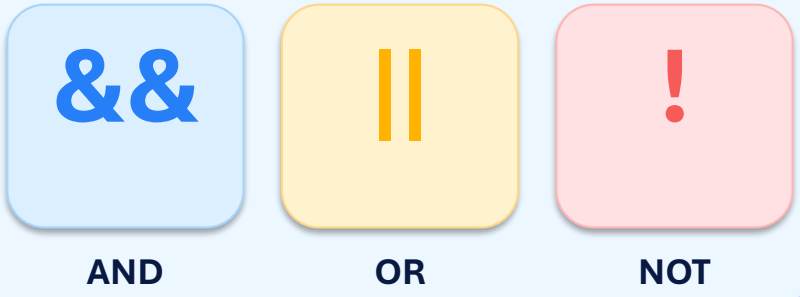
Key Idea | Boolean expressions are the yes-or-no questions your program asks.

Logical Operators

Combining and reversing conditions in JavaScript

1 Main Operators

- && means AND
- || means OR
- ! means NOT



2 How They Work

- AND is true when both conditions are true
- OR is true when at least one condition is true
- NOT reverses true to false and false to true

```
true && false    // false
true || false    // true
!true            // false
```

3 Example

- Check if a student passed and attended
- Use OR for alternative conditions

```
let passed = true;
let attended = false;
console.log(passed && attended);
```

Output: false

Key Idea | Logical operators connect conditions and make decisions more powerful.

Truthy & Falsy Values

What are Truthy & Falsy Values?

In JavaScript, values are treated as true or false when used in a condition. This automatic Boolean conversion lets if statements make decisions without an explicit comparison.

Truthy → true **Falsy → false**

Truthy Values

Values treated as true in a Boolean context:

- Any non-empty string (e.g., "hello")
- Any non-zero number (e.g., 1, -5, 3.14)
- The Boolean value true
- Objects, including {} and []
- Arrays, even empty arrays []
- Functions, e.g., function() {}

Falsy Values

Values treated as false in a Boolean context:

- false (the Boolean value)
- 0 and -0 (zero)
- "" (empty string)
- null
- undefined
- NaN (Not a Number)
- 0n (BigInt zero)

Examples in Conditions

JavaScript converts each condition to a Boolean:

```
if ("Hello")
  console.log("Truthy");
```

→ **Truthy**

```
if (0) console.log("Yes");
else console.log("Falsy");
```

→ **Falsy**